LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 5
Student Name: Gorapalli Sravya
Roll No.: A24126552257
Date: 26-08-2026

---

1. TITLE

Implement and Use Modules in Node.js

2. AIM

To understand and implement modules in Node.js by creating a custom module and using a built-in module.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To create and export a custom module using module.exports.
- To import and use a custom module in another file using require().
- To use a built-in Node.js module (os) to retrieve system information.

4. THEORY

In Node.js, a module is a reusable block of code whose functionality can be exported from one file and used in another. Node.js uses the CommonJS module system, where module.exports is used to expose functionality from a file, and the require() function is used to import it elsewhere.

Node.js also provides several built-in modules that can be used without any installation, such as fs (file system operations), os (operating system information) and path (working with file paths).

Organizing code into modules improves reusability, readability and maintainability, especially as an application grows larger.

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project folder.
2. Initialize the project using npm init.
3. Create a custom module file, calculator.js, containing arithmetic functions.
4. Export the functions from calculator.js using module.exports.
5. Create a main file, app.js, and import the custom module using require("./calculator").
6. Call the imported functions with sample values and log the results.

7. Import the built-in os module and retrieve system information.
8. Run the application using node app.js.
9. Verify the console output.

6. PROGRAM

calculator.js

```
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  return b !== 0 ? a / b : "Cannot divide by zero";
}

module.exports = { add, subtract, multiply, divide };
```

app.js

```
const calculator = require("./calculator");
const os = require("os");

console.log("Addition:", calculator.add(10, 5));
console.log("Subtraction:", calculator.subtract(10, 5));
console.log("Multiplication:", calculator.multiply(10, 5));
console.log("Division:", calculator.divide(10, 5));

console.log("\nSystem Information:");
console.log("Platform:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory (MB):", (os.totalmem() / (1024 * 1024)).toFixed(2));
console.log("Free Memory (MB):", (os.freemem() / (1024 * 1024)).toFixed(2));
```

7. CODE EXPLANATION

Tag / Code	Explanation
module.exports	Exports functions/objects from a module so they can be used elsewhere
require("./calculator")	Imports the custom calculator module
require("os")	Imports the built-in os module
os.platform()	Returns the operating system platform
os.arch()	Returns the CPU architecture
os.totalmem() / os.freemem()	Return the total and free system memory in bytes

console.log()	Prints output to the terminal

8. TEST CASES AND EXECUTION DATA

The application was run from the terminal using node app.js and the console output was verified.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Run node app.js	Console shows addition, subtraction, multiplication and division results	All results displayed correctly	Pass
TC-02	Check division by zero	Displays "Cannot divide by zero"	Message displayed correctly	Pass
TC-03	Check os.platform() output	Displays the correct OS platform	Correct platform displayed	Pass
TC-04	Check memory values	Displays valid memory figures in MB	Valid memory figures displayed	Pass
TC-05	Modify calculator.js and re-run	Updated logic is reflected in the output	Updated logic reflected correctly	Pass

9. OUTPUT

Output: Terminal Execution

[Insert screenshot of the terminal showing the output of node app.js.]

10. RESULT

Thus, a custom module and a built-in module were successfully implemented and used in Node.js, and the module outputs were verified in the console.